DigitalTwin.ai — the three pipelines, in full

Context

The brief asks for a live, data-driven twin that shows where bottlenecks are *forming* and predicts defects *before they happen*, and it explicitly asks what to model versus skip and what to do about stations with little or no sensor data.

This document is the complete design for the three pipelines that answer it. It merges what the two source PDFs got right, what our own experiments measured, and the design decisions worked out in review. Where a document and a measurement disagreed, the measurement won and it is marked.

No code, no repo layout. Pipelines, measurement discipline, and how data physically reaches the system.

Coverage against the brief

CLAUSE OF PROBLEM STATEMENT 4	ANSWERED IN
a live, data-driven **virtual model**	Part 1.4 — the model, its calibration, twin drift and re-fit
helps plant teams **see** where bottlenecks are forming	Part 4.1–4.3 — station record, alert contract, observability map
predict likely defects before they happen	Part 2, whole
a bottleneck **ripples downstream**	Part 1.1 — blocking and starvation waves
a defect **uncaught for dozens of vehicles**	Part 2, Stage 7 — genealogy containment
what the twin must **model versus skip**	Part 5, with ablation discipline
moving from **visualizing** the line to **predicting** problems	Part 4.4 — candidate evaluation
stations with **little or no sensor data**	Part 3, treated as the centrepiece

All eight clauses are addressed. Two — what to model versus skip, and unsensored stations — go beyond what the brief asks.

Part 0 — How data gets from the station to the twin

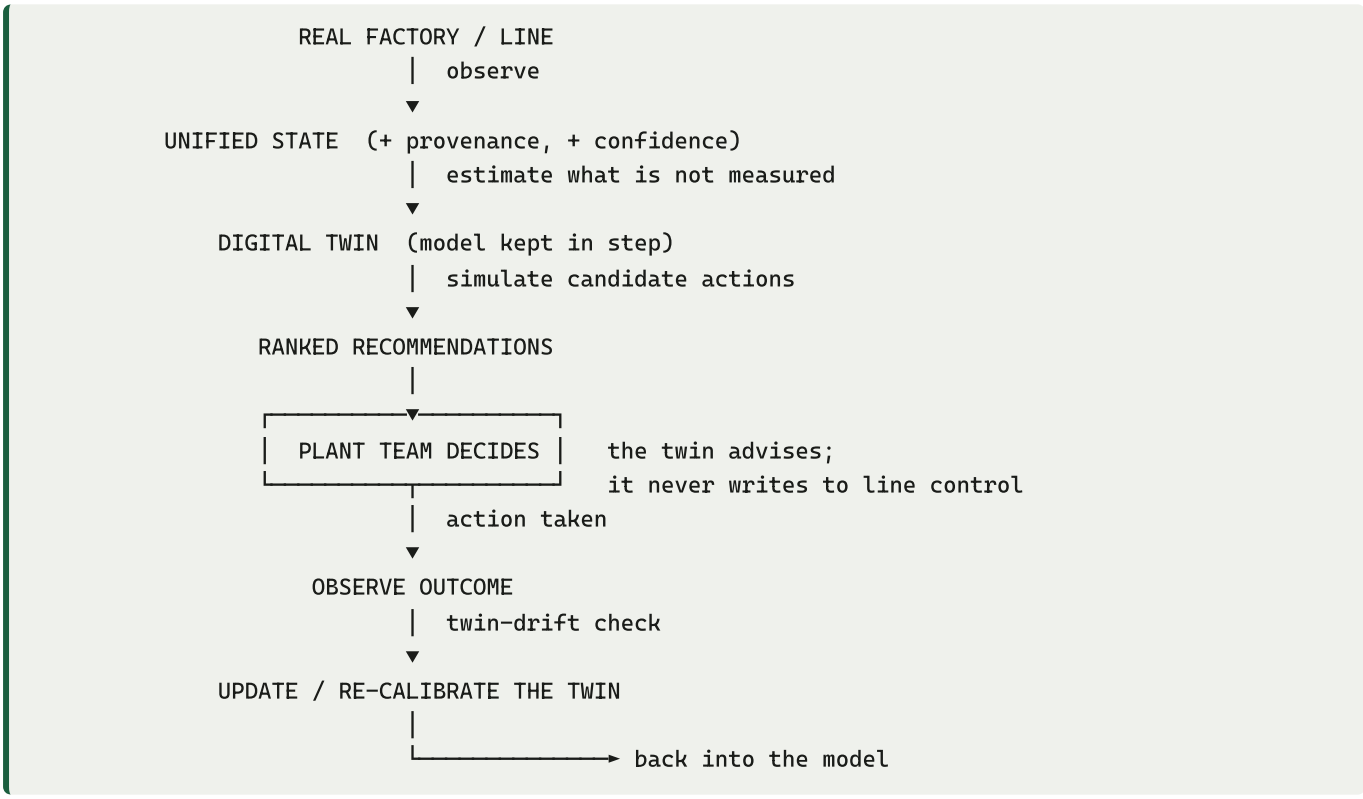
This matters more than it sounds: it is the difference between a proposal a plant can adopt and one that needs a procurement cycle. **Every layer below already exists in a modern plant.** We are subscribing to a stream the plant already produces for other reasons and currently discards.

The physical path

```
Tool / machine controller
        │  Open Protocol (tightening controllers)
        │  OPC-UA        (PLCs, robots)
        │  MTConnect     (machine tools)
        ▼
Edge gateway  — normalises, timestamps, buffers locally
        │  MQTT / Sparkplug B  (store-and-forward: readings survive a network drop)
        ▼
Time-series store  — TimescaleDB / InfluxDB / existing plant historian
        │
        ▼
Streaming detectors  — two state variables per tool, microseconds per reading
        ▼
Unified station state  →  the three pipelines
```

The loop that makes it a twin rather than a report

The brief asks for a **live** model. Everything above is one-directional — data in, analysis out — and a reader would be right to call that offline analytics. What makes it live is that the loop closes:



Two things about this diagram are deliberate.

The human is inside the loop, not beside it. Every closed-loop factory diagram we reviewed runs `recommend → act` as though the system acts. Ours cannot: writing setpoints to line control is a safety-certified change no plant grants a prototype, and a design that quietly assumes it away tells a practitioner we have not spoken to one. The loop closes through a decision, and that is the honest deployable boundary.

The return path is the twin updating itself, not merely logging what happened. Twin drift — divergence between predicted and observed throughput — is what triggers re-calibration, so the model tracks the line as it changes rather than describing the line it was fitted to months ago. That is the difference between a twin and a snapshot.

Why each source already exists

SOURCE	PROTOCOL	WHY THE PLANT ALREADY HAS IT
Tightening controllers (Atlas Copco, Bosch Rexroth, Desoutter, Estic)	Open Protocol	safety-critical joints are **legally traceable** — torque, angle, timestamp, tool ID and result are already recorded for every fastening
PLCs and robots	OPC-UA	required to run the equipment; reading tags is a software subscription, not a hardware purchase
Machine tools	MTConnect	standard read-only machine-state interface
Barcode / RFID scanners	MES	already at station boundaries for VIN traceability
End-of-line test	MES / quality system	standard practice
Andon	andon system	already logged, rarely analysed

Designed to leverage existing plant data interfaces with minimal new sensing hardware. The signals above are already produced for traceability, compliance or control, so the twin subscribes to a stream the plant generates and currently discards rather than commissioning a new one.

That is a claim about *sensing*, not about total cost, and the distinction matters. Real deployment still carries integration effort, edge-gateway provisioning, protocol licences, cybersecurity review, network capacity and historian access, plus plant validation before anything is trusted. Storage is genuinely negligible — 54,000 tool readings for a week is under 5 MB — but storage was never the expensive part.

The five event types the twin consumes

EVENT	FIELDS
`unit_scan`	vin, station_id, event (in/out), timestamp
`station_state`	station_id, state, timestamp, quality code
`buffer_level`	buffer_id, level, capacity, timestamp
`tool_reading`	tool_id, program_id, vin, target/actual torque, target/actual angle, current, temperature, result (OK/NOK), timestamp
`quality_result`	vin, gate_id, result, defect codes, timestamp
`andon`	station_id, reason code, start, duration

(`andon` was listed as a data source but had no event type — corrected. It is the operator's own account of why the line stopped, and it is the only source that carries a human-assigned reason code.)

Boundary scans carry much of the required flow and genealogy information wherever scan coverage is reliable — cycle times, station states, buffer levels, active periods and the build record all follow from them. A plant that offered nothing but `unit_scan` could still support a large part of this twin.

Where that breaks down, and we know because we built the failures ourselves. Rework loops send a unit back through stations it has already visited, so a naive reconstruction double-counts it. Dropped scans leave a unit apparently still inside a station it left. Parallel stations mean two units occupy one logical operation at once. Our own generator injects all three — rework, 0.2% packet loss, and a duplicated station pair in layout L3 — precisely so the reconstruction is tested against them rather than assumed away. Scan coverage is a prerequisite, not a given.

What the path must handle (and most demos ignore)

- **Clock skew.** Controllers are not synchronised. Timestamps arrive offset by seconds. Any logic that assumes a common clock breaks on contact with a real plant.
- **Dropped and duplicated packets.** Store-and-forward helps but does not eliminate it.
- **NULL bursts** during network blips — a channel goes blank for 30 seconds.
- **Late arrivals.** An edge gateway that buffered through an outage delivers a batch of old readings after newer ones.

The twin must degrade gracefully on all four, and a NaN-safe detector must skip a missing reading rather than let it poison a running total.

Part 1 — Pipeline 1: bottleneck formation and its cause

1.1 The real problem, as we measured it

The intuitive model of a bottleneck is wrong in three specific ways, and each one is a slide.

A slowdown at a non-constraint does nothing at all. We jammed station 3 of a five-station line to 300 s, let five cars pile up, repaired it, then recalibrated it to 30 s — twice as fast as every other station. Output: **106 cars either way. Identical.** The queue relocated from in front of station 3 to in front of station 4 in five minutes, and station 3 then sat blocked 30% of the time. *An hour saved at a non-bottleneck is a mirage* (Goldratt, Theory of Constraints, 1984).

A backlog never clears while input continues. To shrink a queue you must work faster than cars arrive. If every station runs at the same rate and cars keep arriving, the pile travels along forever. In simulation the buffer in front of station 4 sat at capacity for the entire remaining run. It emptied only when we stopped admitting new cars — at which point the line drained completely within twelve minutes. **Breaks, shift ends and deliberate release cuts are the only WIP-drain mechanisms that exist.**

The constraint moves. In our runs it changed station about **6 times per shift, up to 25** — and it did so in runs with *no injected fault at all*, purely from buffers filling and draining on a balanced line. A fixed answer of the form "the bottleneck is station 3" is obsolete by lunchtime.

Two consequences worth stating in the pitch:

- **Little's Law** (WIP = throughput × lead time) means the leftover queue is permanent cost: after the jam, WIP was 8 cars higher and every subsequent car spent 8 minutes longer in the plant, at unchanged output.
- **After a disturbance the line is more fragile than before.** A buffer does its job half full — it needs empty space to absorb a surge and stock to cover a gap. Post-jam, the buffer before the constraint is pinned full and those after are pinned empty. Every station is healthy, the dashboard looks fine, and the next disturbance hits harder.

The free lever nobody reaches for: capping cars-in-line (the *rope*, or CONWIP) gave the **same throughput with 36% lower lead time** and zero equipment change.

1.2 What to measure — and what not to

Measure

QUANTITY	WHY
Processing time per unit per station (work only)	the ranking key; the only thing that separates a slow station from a busy one
Availability (uptime ÷ scheduled time)	a 55 s station down 10% of the time effectively needs 61 s
Effective cycle time = processing ÷ availability	the actual constraint ranking
Blocked share, starved share	direction-finding: blocked points downstream, starved points upstream
Buffer level **and its slope**	slope gives a physical countdown to blocking or starvation
Each station's **own baseline** processing time	drift is judged against self, never against takt
Drift rate (seconds of processing time gained per hour)	feeds time-to-overtake
Units completed per window	tells you whether an average is trustworthy yet
Product variant in the build sequence	variant-driven constraint shifts are knowable in advance
Time since last break	post-break warm-up is real and will otherwise look like degradation

Do not measure, or do not act on

AVOID	WHY
Utilisation as a ranking signal	*correction to both PDFs.* A station fed heavily and a station that is intrinsically slow both read ~95% busy. Utilisation cannot separate them. Keep it as a displayed KPI; never rank by it.
Dwell time as if it were processing time	dwell includes waiting caused by a full buffer downstream; charging that to the station blames a victim
Single-car thresholds ("this car took >60 s")	manual station times scatter widely — published coefficient of variation **0.25–0.6**, so a 68 s car on a 54 s station is normal noise, several times an hour
Comparison against takt	takt is a line target. On a healthy line where every station is under takt, one of them is *still* the constraint. Nothing crosses a threshold and you detect nothing.
3D geometry, robot joint trajectories, weld/paint physics	zero predictive value; collapse into a cycle-time distribution
Individual operator performance	measure the station, never the person

The sample-size rule (from break 3)

Averaging shrinks noise by √n. On a station with ±8 s spread:

SAMPLES AVERAGED	REMAINING NOISE
1 car	±8.0 s
4 cars	±4.0 s
20 cars	±1.8 s
100 cars	±0.8 s

To catch a shift of a given size you need roughly: **1σ shift → ~10 cars; 0.5σ → ~40; 0.25σ → ~150.** A fixed window forces a bad trade — short catches big shifts fast and misses small ones; long catches small shifts slowly. **CUSUM removes the choice**: it accumulates evidence and fires as soon as there is enough, fast for large shifts and patient for small ones. You set a sensitivity, not a window, and calibrate it on healthy data until the false-alarm rate is acceptable.

Station type changes the method. A robotic station's cycle times are extremely tight (one study: 5th-to-95th percentile spread of **0.94 seconds** across 194 cycles) — a simple threshold works there. A manual station needs drift detection. The plant knows which stations are which, so use both.

1.3 The pipeline

Stage 1 — Estimate station state from boundary scans. For every station and every second, estimate working / blocked / starved / down. If PLC state tags exist, read them — then the state is observed. If not, it is **inferred, and a 60-second interval is genuinely ambiguous**: it can be work plus waiting, work plus a micro-stop, or work plus blocking. Output a **belief over states** (Working 0.72 / Blocked 0.21 / Micro-stop 0.07), not a hard label, and carry that uncertainty forward. The neighbour signature is what sharpens the belief:

> *If station 2 is the constraint, station 1 shows its **in-to-out interval stretching past its own baseline** (finished, cannot hand over → blocked), and station 3 shows **gaps between arrivals** (nothing reaching it → starved). Station 2 is the one with elevated dwell whose neighbours show that signature.*

This works from barcode scans alone. It fails only when several consecutive stations are dark, which is Pipeline 3's Tier C.

Stage 2 — Compute processing time, not dwell. Subtract blocked and starved time.

Stage 3 — Divide by availability → effective cycle time. This is the ranking key.

Stated explicitly because it invites a double-counting question: processing time counts only time in the `working` state, so blocked, starved **and down** are all excluded from the numerator. Availability then prices down time once, in the denominator, as `(scheduled − down) ÷ scheduled`. So `effective = processing ÷ availability` reads "*55 s of work at 90% availability effectively needs 61 s per car.*" Down time is removed from the numerator and charged once in the denominator — **not twice**. Blocked and starved are excluded from processing and correctly absent from availability, because they are not the station's own fault.

Stage 4 — Baseline each station against itself. Learn the healthy mean and spread from an in-control window. Everything afterwards is judged against that, not against takt.

Stage 5 — Accumulate evidence. Run CUSUM on processing time per station. Never alarm on a single car.

Stage 6 — Rank into CANDIDATES, not a verdict. Highest effective cycle time is the **leading candidate constraint**, not the constraint by definition. Two reasons it can be wrong: a station may be slow *and* frequently starved, in which case speeding it produces nothing because it spends its time waiting anyway — and our availability term penalises downtime but **not** starvation. So rank on effective cycle time, then down-weight candidates carrying high starved share, and emit a **ranked pair** with a margin.

The definitive answer is economic — the station whose speed-up actually produces more cars. That is measurable **offline, in the simulator**, and it is how we validate the detector. It is *not* available in a live plant, which cannot be re-run under identical conditions. Live, the twin commits to a ranked candidate with confidence; offline, we score that candidate against counterfactual truth. Keep those two roles separate.

Stage 7 — Cross-check. The constraint should also be the station least often forced idle by a neighbour. Agreement raises confidence; disagreement is reported, not hidden.

Stage 8 — Risk of becoming the constraint (this is "forming"). `gap ÷ drift rate` is the intuition, but **do not ship it as an exact countdown.** It assumes drift is stable and linear, and real drift is noisy, intermittent and nonlinear — disturbed by variants, breaks, maintenance and micro-stops. We already learned this the hard way on the quality side: fitting a curve to the early flat stretch of a degradation made our remaining-life estimates **~3× too long**. The same extrapolation error applies here.

So report **P(station i becomes the constraint within horizon H)** for H ∈ {15, 30, 60} min, built from the drift estimate *and its uncertainty*, the gap, buffer trajectory and the known variant sequence. Quote a point countdown only if experiment shows it is reliable.

> ***Measured — and it does not work.*** *We built this and scored it. When the model states 70–100% confidence, the station actually becomes the constraint* ***5.9%*** *of the time; at 50–70% stated it is right 33%. The probabilistic form is still badly overconfident.* ***This is the second time linear extrapolation of an early trend has failed in this project*** *— remaining-life estimates were ~3× too long for the same reason. Drift observed over a few windows does not predict where the constraint moves, because the constraint is set by the whole coupled system rather than by one station's trajectory. Do not present drift-based overtake risk as a working capability. The honest statement is that we tested it, measured it, and it failed — which is more useful to a jury than a capability nobody checked.*

Two distinct flags, two meanings: *drifting* (abnormal for itself, no production impact yet) and *elevated risk of constraining within H* (actionable).

Stage 9 — Buffer countdown, stated conditionally. `slots remaining ÷ fill rate` = minutes until the upstream station blocks. Pure arithmetic, no training data, works on day one at a plant with no history — the cheapest answer to "forming".

Label it honestly: **"projected blocking horizon under current flow conditions."** It is a nowcast, not a forecast. A variant change, downstream recovery, a break or an intervention all invalidate the fill rate it was computed from.

> ***Measured, and this one works — with a stated limit.*** *Across 178 forward-looking warnings, the station behind the buffer went on to block within the hour* ***60%*** *of the time. The countdown is* ***unbiased in the middle*** *— median error* ***+0.6 min****, mean* ***+0.4 min*** *— but scattered at the edges: the interquartile range is* ***−11.9 to +10.6 min****, and only 11–33% of individual predictions land within ±5 minutes. So it is a* ***ranked early-warning ordering, not a precise countdown*** *— which is exactly what this stage already committed to claiming, now backed by measurement rather than caution. Use it to decide which buffer to watch, never to promise when. Note also what it is not: a buffer already at capacity is excluded, because "0 minutes to full" is a statement about the present, not a warning. Before that filter was added, 557 of 558 outputs were already-full buffers and the stage looked productive while predicting nothing.*

Stage 10 — Persistence filter. Only alert on a constraint predicted to hold long enough for a human to act. Below roughly 10 minutes it has moved before anyone arrives. This is what stops the system becoming whack-a-mole.

Stage 11 — Cause diagnosis, narrowed first. Only now pull that one station's equipment and process data: tool telemetry, temperature, vibration, maintenance history, cycles since service, variant, batch, shift, neighbour states. Report a **likely** cause with evidence and confidence — never a causal claim.

(This is the cascade principle, and it is the best idea in the source PDFs: cheap checks on everything, expensive checks only on suspicion.)

Stage 12 — Decide, and simulate before acting. Candidate actions compared against the same starting state with paired replications:

OPTION	MUST BE IN THE LIST BECAUSE
Do nothing	often correct; the constraint may move on its own
Reduce the release rate	the free lever — same throughput, 36% lower lead time
Move a floating operator to the constraint	classic, and only works *at* the constraint
Pre-stage material at the threatened station	cheap and fast
Re-sequence variants away from the constraint	the MES already knows the build order
Bring forward a planned micro-stop	converts unplanned loss into planned
~~Speed up the slow station~~	**include it to show it recovers nothing**

Report a distribution across replications, not one lucky run.

1.4 The virtual model, and how it stays in step

Everything above is arithmetic and statistics over an event stream. That is deliberate and it is most of the value — but on its own it is an *estimator*, not a twin. Under the Kritzinger taxonomy, what makes a system a digital twin rather than a digital shadow is a **virtual model kept in step with the physical line**. This is that component, and without it a reader could correctly conclude we built a stream-analytics service.

What the model is. A discrete-event model of the line: per-station processing-time distributions, buffer capacities, failure and repair processes, variant routing, and the shift calendar. It runs *alongside* the line, not instead of it.

How it is calibrated — from the same stream the detectors already consume.

MODEL PARAMETER	FITTED FROM
processing-time distribution per station	boundary scan pairs, minus waiting
buffer capacity and behaviour	buffer level events
failure and repair processes	state transitions into and out of `down`
variant time multipliers	scan times regressed on the MES build sequence
calendar effects	break windows and post-break warm-up

No parameter requires data a plant does not already emit. That is what keeps the minimal-new-sensing claim true for the twin itself, not merely for the detectors.

Twin drift — the health check on the twin. Divergence between model-predicted and observed throughput over a trailing window. This is the metric that tells you the model has stopped being true, and the design had no such metric before. A model nobody re-fits quietly stops describing the line, and then every recommendation it produces is wrong in a way nothing detects.

Re-calibration trigger. When twin drift exceeds threshold, re-fit and **log the event**. Changeovers, layout changes, tooling replacement and maintenance are expected triggers. The re-fit is logged because a silently re-fitted model cannot be audited after an incident.

What the model is used for — and what it is not.

USED FOR	NOT USED FOR
candidate evaluation — ranking interventions under common random numbers (Stage 12)	writing setpoints or schedules to line control
twin inversion for opaque dark blocks (Pipeline 3)	any closed-loop action
generating the economic ground truth that validates the detectors offline	replacing the measured state where measurement exists

The exclusion is not modesty. Writing to safety-certified line control is a regulated change no plant grants a prototype, and a design that claims it tells a practitioner we have not spoken to one.

1.5 The design corrections, and where they came from

CORRECTION	SOURCE
Rank by processing time, not utilisation	measured: utilisation ranks a busy station and a slow station identically
Divide by availability	a 55 s station down 10% is worse than a reliable 58 s one
Compare to own baseline, not takt	a line where every station is under takt still has a constraint
Never threshold on one car	manual CV 0.25–0.6; a 68 s car on a 54 s station is noise
Predict transitions, not a static location	the constraint moves ~6× per shift with no fault present
Include "do nothing" and "slow the input" in the options	speeding a non-constraint measured 106 vs 106 cars

Part 2 — Pipeline 2: defect prediction

2.1 The real problem

Tools do not fail, they drift. A torque tool slowly loses calibration; the vehicle looks correct, the station reports no fault, the operator sees nothing. At a 60 s takt with final test forty stations downstream, the defect surfaces roughly **forty vehicles too late**, each needing teardown and rework. A system that inspects finished products can only ever detect the consequence. A system that watches process parameters detects the cause, and the gap between those two moments is the entire value of the product.

And the instrument you would naturally watch can be the thing that broke. In our data a sensor-bias fault moved the torque *reading* by −0.55σ while true clamp force collapsed −3.74σ. The controller stamped **11,631 genuinely defective joints as OK**. Torque monitoring is structurally blind to that failure class because it is watching the channel that is lying.

The inverse fault costs the opposite way. In a pure transducer drift the tool is perfectly healthy — true torque moved **+0.01σ**, every secondary channel stayed inside **±0.11σ** — but the reading collapsed 3σ, so the controller **falsely rejected 700–960 good joints per tool.** Same alarm. Opposite economics. Opposite fix.

An alarm on one tool may not be about that tool. When a bad fastener lot entered the line, **21 of 23 tools consuming that lot alarmed within 50 operations of each other, and zero tools on other lots alarmed at all.**

A tool fault moves one tool; a material fault moves every tool sharing the part number, simultaneously. Servicing those tools would have been the wrong answer twenty-one times.

And the largest loss is never logged. Measured with automated capture, breakdowns are roughly **18%** of production loss while minor stops are **34%** and reduced speed **22%**. Micro-stops under five minutes clear before anyone raises a ticket — an estimated 15–25% of capacity per shift, invisible in the plant's own downtime report. They are detectable for free as anomalous dwell between two boundary scans.

2.2 What to measure — and what not to

Know which truth regime you are in

This determines what can be claimed, and it is the sharpest limitation on transferring our results to a plant.

REGIME	WHAT IS AVAILABLE	WHAT CAN BE MEASURED
Simulator	true physical value, true defect, true injected fault, all independent of the sensor	lead time, sensor-bias escapes, false-certification rate — everything
Real plant	sensor reading, controller OK/NOK, sometimes final inspection	only what the instruments say — and **if the faulty sensor generates the label, a model trained on it learns the sensor's error**

Our headline sensor-bias finding — 11,631 defects passed as OK — is *measurable only because we hold independent physical truth*. In a plant that number surfaces as warranty claims months later, not as a metric. So: report it as a simulator result demonstrating a failure mode, and on real data lean on the **cross-channel disagreement** (torque flat while current and angle move), which needs no independent truth to compute.

Measure

QUANTITY	WHY
Every channel the controller emits — torque, angle, motor current, temperature, cycle time	the spec channel can be the one that is lying
The **true** spec limits per joint	defect truth is defined on the applied value, not the reading
The controller's **OK/NOK verdict separately** from the measured value	the gap between them *is* the sensor-fault signal
Baseline mean and spread per channel per tool	drift is relative to self
Spread ratio, not just mean shift	bearing wear widens the distribution without moving the mean
Cross-tool correlation over the **MES fastener-lot record**	separates material faults from tool faults
Genealogy: vin → station → tool → lot → timestamp	turns an alarm into a containment list
Maintenance and calibration events	so a service reset is not mistaken for a fault
Ambient temperature and post-break warm-up	shared confounds that will otherwise look like degradation

Do not measure, or do not act on

AVOID	WHY
The spec channel alone	*correction to both PDFs.* Misses sensor faults entirely — 11,631 joints in our data
Thresholds calibrated on the evaluation data	tuning until zero alarms appear, then reporting "zero false alarms", proves nothing. Ours was really ~1 in 5 tool-weeks
A precise remaining-life countdown	our RUL estimates were biased about **3× too long**, because wear accelerates faster than any curve fitted to its early flat stretch. Show a coarse band — "needs attention today"
Predicting sudden step failures	physics, not a modelling gap. You can predict a tyre wearing bald; you cannot predict a nail. Say so
Raw high-frequency streams into one large model	compute compact features (mean, slope, spread, CUSUM total) and escalate only on suspicion
Attributing quality to named operators	station-level aggregation only

---

2.3 The pipeline

Stage 1 — Baseline every channel per tool from an in-control window.

Stage 2 — Calibrate thresholds on held-out healthy tools. Different tools, never used in evaluation. Non-negotiable — this is the difference between a measured false-alarm rate and a defined one.

Stage 3 — Cheap continuous monitoring. CUSUM and EWMA on every channel of every tool. Two state variables per channel, microseconds per reading, no batch job.

Check autocorrelation before you calibrate. Standard CUSUM and EWMA control limits assume independent observations. Industrial measurements frequently are not — consecutive fastenings share a joint, an operator, a lot, an ambient state. Autocorrelation inflates the false-alarm rate badly and silently. Before setting any threshold, inspect sampling rate, autocorrelation, operating modes, missingness, variant and warm-up/maintenance resets; if residual autocorrelation is material, model it out (fit and monitor residuals) rather than pretending it is absent. Our empirical held-out calibration partially absorbs this, but absorbing an effect is not the same as knowing its size.

Stage 4 — Station health score → escalate. Aggregate per-channel evidence into one score. Below threshold, spend nothing. Above it, escalate. (Cascade.)

Stage 5 — Classify the failure mode. An alarm is not a work order:

MODE	READING	REALITY	SECONDARIES	CORRECT ACTION
Mechanical wear	−3.28σ	−3.28σ	all move together	service the tool
Sensor bias	−0.55σ	**−3.74σ**	all move	urgent — it is passing bad joints as good
Pure transducer drift	−2.99σ	**+0.01σ**	flat ±0.11σ	**recalibrate only — the tool is fine**
Bearing / spread-only	−0.06σ	−0.06σ	spread ×2.6	replace — calibration is useless

Three features separate all four: primary shift, secondary shift, spread ratio. **The secondaries are the discriminant** — they move for real degradation and stay flat for a sensor fault.

Classify on relationships, not on these exact sigma values. The numbers above come from one generator; a classifier fitted to them would learn our simulator rather than the physics. Use **ratios and correlations** that survive a change of scale — the primary-to-secondary shift ratio separates the modes cleanly (0.87 for wear, 0.17

for sensor bias, 0.01 for spread-only) and is invariant to the absolute drift size. Then test on a generator whose damage mechanism was never used in development; we already hold one (Wiener-process degradation) for exactly this.

Stage 6 — Cross-tool lot check. Before writing a work order, ask whether every other tool on the same lot alarmed at the same time.

State it as a **likely common cause, not proven causation** — shift change, ambient swing, a software deployment or batch maintenance could also make many tools move together. What raises this well above bare correlation is the **negative control that falls out of the lot structure**: in our data 21 of 23 tools on the bad lot alarmed and **zero tools on other lots did**. A shift or ambient effect would have moved those too. So the recommended action is *hold and inspect the lot*, with the confounders listed and the control stated, rather than a bare quarantine order.

Stage 7 — Defect risk and genealogy. Produce the containment list: "*station 22 has been drifting since 10:14; these 38 vehicles were built on it since.*"

Stage 8 — Stop or continue. The joint decision that needs both pipelines:

```
cost of stopping now  = downtime × JPH × margin × P(this station is the constraint)

cost of continuing    = ∫ h(u) du over units to the next window × rework cost
                        + P(escape) × field failure cost
                        where h(u) is the per-unit defect hazard, which RISES
```

Two things here are deliberate and were wrong in an earlier draft.

The constraint term is a probability, not an indicator. Pipeline 1 emits a ranked candidate with a confidence and explicitly refuses to emit a verdict. A hard `1{is the constraint}` would silently reintroduce the certainty the rest of the design removes — and would make the decision flip on a coin-toss between two near-tied stations. Weighting by P(constraint) makes the cost degrade smoothly as the evidence weakens, which is the honest behaviour when the line is balanced and nothing dominates.

The defect term integrates a rising hazard, not a constant rate. The entire premise of this pipeline is that degradation *accelerates* — a tool two hours from making scrap is far more dangerous per unit than one twenty hours out. Multiplying a fixed P(defect) by the unit count contradicts the physics the detector is built on and understates the cost of waiting exactly when waiting is most expensive.

The decision that falls out:

- **Low P(constraint)** → stopping costs almost nothing, buffers absorb it. **Fix now.**
- **High P(constraint)** → every minute is one car. Usually wait for the next planned stop, where the cost falls back to near zero — unless the integrated hazard over that wait exceeds it.

The same drifting tool gets **opposite correct answers** depending on flow state.

Stage 9 — Verify, then re-arm. Confirm the readings returned to target after the technician acted — almost nobody closes this loop and it is nearly free. Then **reset the detector**. A first-alarm detector latches: in our data one material-lot event masked every subsequent tool fault on those tools. Without acknowledge-and-rearm, one bad batch blinds the system for a week.

Part 3 — Pipeline 3: stations with little or no sensor data

3.1 The real problem

Manual stations, legacy equipment and controllers that expose no useful tags are normal, not exceptional. Most teams will treat this as a limitation to acknowledge. **It should be the centrepiece**, because it is the condition most real lines are actually in.

The governing principle, and the best line in the source PDFs: **do not hallucinate missing readings.** Estimate useful state from available evidence and attach confidence and provenance. A directly measured value and an inferred value must never look identical in the interface.

3.2 What to measure — and what not to

Measure

QUANTITY	WHY
Observability per station, as a **percentage**	"2 of 10 physical signals, 6 virtual estimates, 81% confidence" — not a yes/no
Upstream exit and downstream entry timestamps	bracket the dark station exactly
Known transit time between stations	subtract it to isolate occupancy
Neighbour states (blocked / starved)	the only way to split work from waiting
Buffer levels either side	flow conservation: upstream queue growing + downstream arrivals falling ⇒ restriction between them
Product variant per unit	variant regression attributes block time to specific stations
Arrival-delay difference at the two bracketing sensors	locates a constraint inside a dark block

Do not

AVOID	WHY
Treating bracketed time as processing time	*correction to both PDFs.* Their worked example — "70 s observed − 10 s transit = 60 s hidden station time" — is **occupancy = work + waiting**. If the dark station was starved or blocked you have blamed it for a neighbour's delay and sent a technician to the wrong place
Forcing per-station attribution inside a consecutive dark block	one measurement cannot be split into three unknowns; report segment-level state with low attribution confidence
Inventing a plausible sensor value	a fabricated reading is worse than an honest gap
Asserting confidence percentages	they must be calibrated against known truth, not chosen

3.3 The pipeline

Stage 1 — Build the observability map. For every station, what does it actually emit.

Stage 2 — Classify into tiers. Resolution degrades; the system never fails.

TIER	SITUATION	METHOD	RESULT
A	fully instrumented	processing time + cross-check	**directly observed**
B	one dark station between two scanned	bracketing **+ state estimator**	**strongly estimated** (not exact)
C	several consecutive dark	localise, then propagation delay + variant regression	**segment-level localisation**
D	fully opaque section	flag + costed sensor-placement recommendation	bounded

Tier B is not exact, and calling it exact contradicts our own data generator. We deliberately inject per-controller clock skew of ±3 s, dropped packets and NULL bursts — then claiming a bracketed estimate is exact is internally inconsistent. Variable transit time, micro-stops inside the dark station and late packets all add uncertainty. Treat **transit time as a distribution**, not a constant, and propagate its spread into the hidden-state estimate.

Stage 3 — Tier B: bracket, then split. `occupancy = downstream entry − upstream exit − transit`. Then use the neighbours' observed states to separate work from waiting: if the upstream buffer was empty, the dark station was starved and the gap is not its fault. **Bracketing and state estimation are one mechanism, not two options.**

Stage 4 — Tier C: localise before resolving. If the block's total time exceeds every instrumented station's time, the constraint is *inside* the block. **Twenty stations narrowed to three is already most of the operational value** — a supervisor walks three stations, not twenty. It needs no model.

> ***Built, but currently unvalidatable — and the reason is a dataset gap, not a verdict on the method.*** *Only* ***5 of 142*** *dark blocks in our data actually contain the constraint. A detector that always answered "elsewhere" would score 96% on exoneration purely from that base rate, so no threshold can be calibrated or meaningfully scored. The cause: dark stations are drawn at random while the constraint concentrates on the few genuinely slow stations, so the two rarely coincide — 3.5% observed against 15% if placement were uniform. The fix is to bias dark-station selection toward the constraint in a share of runs so positive cases exist to learn from. Also worth recording: two earlier versions of the comparison were wrong in opposite directions. Comparing block traversal time against station processing time made the test fire on 84% of blocks; correcting to station-to-station intervals made it fire on almost none. The quantity on each side of that inequality has to be the same quantity, and getting it wrong is easy — which is itself an argument for scoring the method rather than trusting it.*

For position within the block:

- **Propagation delay.** A slow station pushes a blocking wave backwards and a starvation wave forwards, each travelling one buffer at a time. The *ratio* of arrival delays at the two bracketing sensors locates the source — the same principle that places an earthquake epicentre from arrival times.
- **Variant regression.** Different variants load stations differently and the MES records each unit's variant — thousands of free natural experiments the plant is already running.
- **Twin inversion.** Tune hidden station parameters until simulated observables match reality. Most general, most expensive — and **not uniquely identifiable**: several hidden configurations can reproduce the same observations. Return **ranked hypotheses with posteriors**, never one confident configuration.

Where several hidden stations change at once, exact attribution may be impossible in principle. The honest output is then: "*abnormality is inside S05–S07; S06 most likely; attribution confidence low.*" That is a useful answer. A confident wrong station is not.

Stage 5 — Tier D: turn the gap into a purchase recommendation. *Given a budget of k sensors, place them here to maximise localisation power, weighted by each station's prior probability of constraining the line — and here is the observability gained per sensor.* This changes the conversation from "instrument everything" to "spend this much, here, for this much diagnostic power".

Stage 6 — Confidence, provenance, and validation. Every inferred value carries the estimate, a confidence, and the evidence that produced it. Maintain a **belief distribution** (Normal 72% / Blocked 21% / Down 7%) rather than "UNKNOWN". Validate by **switching off sensors we actually have**, estimating anyway, and comparing to known truth — when the system says 90%, check it is right about 90% of the time. That is also the strongest thirty seconds of the demo.

Part 4 — The decision surface: what the twin emits

The three pipelines above end at computed state. That is not where the brief ends. It asks for a twin that **"helps plant teams see where bottlenecks are forming"** and asks how the system moves **"from visualizing the line to predicting problems."** A pipeline that stops at "we computed a ranked candidate" has not specified its output.

This part specifies **what leaves the system and with what guarantees**. It is a contract, not a screen design — what must accompany a value, and what disqualifies an alert.

4.1 The station record

One row per station per interval. Every field carries three things, and **a field missing any of them is not emitted**:

value	the number itself
provenance	`measured` / `inferred` / `predicted`
confidence	calibrated, not asserted — validated by switching off sensors we have and comparing to known truth

This is the mechanism that turns "*a measured value and an inferred value must never look identical*" from a good intention into something enforceable. If the twin cannot say where a number came from, the twin does not publish that number.

4.2 The alert contract

An alert may be raised **only** if it carries all of:

1. the ranked candidate and its **margin** over the next candidate
2. the **evidence** that produced it — which signals moved, and by how much
3. a **persistence estimate**: will this last long enough to be worth walking to
4. the **recommended action**
5. the **expected cost of not acting**

An alert that cannot state its evidence is suppressed rather than downgraded. This is the rule that stops the system becoming whack-a-mole against a constraint that moves roughly six times a shift, and it is why the persistence filter (Stage 10) exists upstream of it.

4.3 The observability map as an output

Not an internal diagnostic — a first-class thing the plant sees. Per station: how much is measured, how much inferred, at what confidence, and for Tier D the **costed sensor recommendation**. The brief's hardest question deserves an output, not merely a method.

This is also the honest face of the system: a station reading *"2 of 10 signals measured, 6 inferred, confidence 0.81"* tells an operator exactly how much to trust what follows.

4.4 Candidate evaluation — this is "visualizing → predicting"

> ***A word we use in exactly one sense.*** *Counterfactual is reserved for offline evaluation only — re-running a recorded shift under identical random draws with one station faster, to score the detector against a truth it cannot recompute. It is never used about live operation, because a live plant cannot be re-run and claiming otherwise would imply we know what would have happened. Live, the twin performs* ***candidate evaluation****: it simulates actions it has not taken, ranks them, and then learns from the outcome that actually occurs.*

For any active constraint candidate, the twin emits the **ranked intervention comparison**: each candidate action with its predicted recovered throughput and the uncertainty on that prediction, evaluated under common random numbers so the comparison is paired rather than lucky.

The list must include **do nothing** and **reduce the release rate**, because both are frequently correct and neither is anyone's instinct.

This is the literal answer to the brief's third prompt. A dashboard shows what happened. A twin shows **what would happen under each action available right now** — including that the obvious action, speeding up the slow station, recovers nothing when that station is not the constraint.

4.5 Replay as a system property

The decision surface must be reconstructable from a stored event log at arbitrary speed.

This matters beyond demonstrations: it is what makes the system **auditable**. After an incident the only question that matters is *what did the twin know, and when did it know it* — and that question is unanswerable unless the surface can be replayed exactly as it stood at that moment.

Part 5 — What the twin models, and what it skips

The brief asks this directly. The rule: **model what propagates on the timescale of your decision and can actually be observed; collapse everything else into a distribution.**

Model

ELEMENT	WHY IT SURVIVES THE TEST
Station processing-time distributions, including the tail	propagates through buffers; the tail is where constraints are born
Buffer capacities and levels	the coupling mechanism — without buffers there is no blocking or starving, and therefore no bottleneck
Failure and repair processes	propagates, and creates the constraint while it lasts
Product variants and routing	different loads per station → the shifting bottleneck
Station states over time	the raw material of detection
Tool and process parameters	the only place defect precursors live
Genealogy per unit	turns a prediction into an action
Rework loops	failed units re-entering materially change WIP
Shift calendar (breaks, lunch)	**the only mechanism that drains a backlog**
Material lots	a shared latent that couples tools with no physical connection

Skip

ELEMENT	WHY
3D geometry, CAD-accurate visuals	zero predictive value; the most common way to waste four weeks
Robot kinematics and joint motion	collapses cleanly into a cycle-time distribution
Physics of welding, painting, curing	model the outcome distribution, not the mechanism
Operator ergonomics and individual performance	not needed for throughput, and ethically fraught
Closed-loop write-back to line control	a safety-certified change no plant grants a prototype
Graph neural network over station topology	**cut deliberately, not omitted.** It was always framed as a layout-transfer *experiment*, first to cut if the schedule slipped. Our literature review found no published evidence that graph models beat gradient boosting on a near-serial line, which is what a 20-station spine with one merge is. It stays cut until the core detection and forecasting layers are measured; reinstating it would be a transfer experiment, never the accuracy bet

Prove the boundary, do not assert it. Whether something propagates is an empirical question — we got it wrong twice. The shift calendar looked cosmetic until we measured that a backlog *never* clears without a break; the material lot table looked like flavour until it produced 21 simultaneous alarms. So demonstrate each scoping decision by **ablation**: remove the rework loop, re-run, report what changed.

Part 6 — How we prove any of it

1. **Own the plant.** Because the simulated line is ours, the fault-injection time T0 is known exactly. That is the only reason lead time is measurable rather than assertable.
2. **Fault-injection timeline:** T0 injected → T1 twin alerts → T2 conventional KPI would show it → T3 end-of-line catches it. **Lead time = T2 − T1.** Pin T2 to a fixed convention (15-minute KPI refresh, hourly OEE roll-up) so the baseline is not self-graded.
3. **Ground truth must be economic, not procedural.** The constraint is the station whose speed-up actually produces more cars, measured by re-running with that station faster under identical random draws. **Never label with the same formula the detector uses** — we did that once and scored 95.2%, which was an identity, not a measurement.
4. **Calibrate on held-out data**, and run matched fault-free trials to measure false alarms. A detector with no false-alarm number is not evaluated.
5. **Beat real baselines:** utilisation ranking, fixed-threshold alarms, end-of-line detection, detection-without-forecast, and random alerting at matched rate.
6. **Report cars lost, not just accuracy.** If our pick is worth 4 cars and the best is worth 5, we lost one — which a naming-accuracy metric records as total failure. The operationally honest metric is regret.
7. **Firewall the headline.** Report final numbers on a generator with different failure mechanisms than the one used during development.

Part 7 — What we will not claim

WILL NOT SAY	BECAUSE
A single bottleneck accuracy number covering all conditions	it would average two different regimes into a figure that describes neither. Quote the split: on blocks where a real constraint exists (margin ≥ 2 cars) active period reaches **46% top-1 / 58% top-2** and effective cycle time **43% / 59%**, against **35% / 48%** for utilisation ranking. On a balanced healthy line ours fall to **13–15%** and utilisation to **8.6%** — the correct answer to a question with no dominant answer, and the gap widens precisely where the question is hardest
"Guaranteed false-alarm rate"	conformal guarantees assume exchangeability, which autocorrelated production data violates. Report empirical rates with intervals
That we invented the detection primitives	CUSUM is 1954, active-period is 2001, virtual metrology is standard in semiconductor fabs, Theory of Constraints is 1984. Naming them correctly is what makes the rest credible
A precise remaining-life countdown	measured 3× biased long
That we can predict sudden failures	physics, and saying so builds trust
Closed-loop control of a live line	tells a practitioner you have not spoken to one
That a bracketed dark-station time is **exact**	clock skew, variable transit and micro-stops make it an estimate — and we inject clock skew into our own data, so claiming exactness contradicts our generator
That the constraint **is** the highest effective cycle time	that is the leading *candidate*; the definitive answer is economic and only obtainable offline
That simultaneous lot alarms **prove** the lot is at fault	strong common-cause evidence with a negative control, still not proven causation
A point countdown to a station becoming the constraint	drift is noisy and nonlinear; report risk over a horizon

The sentence to say out loud: "*The detection primitives are established — Roser, CUSUM, virtual metrology. Our contribution is three things: locating constraints inside unsensored blocks, a joint quality-throughput stopping decision that no existing system can make because flow and quality live in separate tools, and a measured lead-time curve where the published record has almost none.*"

Part 8 — Provenance: measured, or cited

Every quantitative claim in this document is one of two things, and the distinction is the first thing a jury will probe. Nothing is unattributed.

Ours — measured, with the code that produced it

CLAIM	SOURCE
106 cars either way when the slow station is doubled in speed	recovery-scenario simulations
same throughput, 36% lower lead time under a WIP cap	release-rate control experiment
constraint changes station ~6× per shift with no fault present	per-minute constraint labelling over simulated shifts
11,631 defective joints passed as OK under sensor bias	v3 process telemetry validation
700–960 good joints falsely rejected under pure transducer drift	same
21 of 23 tools on a bad lot alarm together, 0 on other lots	v3 process telemetry, lot simultaneity check
warning of **421–1,322 vehicles** (per-condition medians), held-out calibration — **except bearing/spread-only faults, median −292, detected after scrap begins**	v3 detection experiment, `results/process_v3_validation.csv`
remaining-life estimates ~3× biased long	v3 RUL convergence study
four failure modes separable on three features	v3 channel-signature analysis
dark-station occupancy 88.8 s vs 58.0 s true work	v5 validation, bracketing-bias probe
faulted runs: top gain 18.0 cars, margin 12.0 (healthy: 7.0 / 3.8)	v5 sensitivity gate check
detection on strong constraints: active period 46% top-1 / 58% top-2; effective CT 43% / 59%; utilisation 35% / 48%	v5 detector evaluation, 958 scored blocks
regret: ours **1.31** cars/block vs utilisation **1.48**, against a 2.27-car ceiling (all scored blocks)	same — ~42% of available gain captured
buffer countdown: 60% of warnings followed by a real block; median error +0.6 min, IQR −11.9 to +10.6	`results/forming_buffer_countdown.csv`
overtake risk: 70–100% stated confidence → 5.9% actual outcome rate	`results/forming_overtake_risk.csv`
dark localisation: only 5 of 142 blocks contain the constraint, so the test cannot be calibrated	`results/dark_localisation.csv`

Cited — from the literature, not from us

CLAIM	SOURCE
robotic station 5th–95th percentile spread of 0.94 s over 194 cycles	published robotic assembly cycle-time study
manual assembly coefficient of variation 0.25–0.6	published assembly task-time variability research
loss split: breakdowns ~18%, minor stops ~34%, reduced speed ~22%	industry OEE loss taxonomy, automated capture
micro-stops 15–25% of capacity per shift	industry micro-stop analyses
automotive unplanned downtime above $2M/hour	Senseye/Siemens, True Cost of Downtime
active period method; shifting bottleneck	Roser, Nakano & Tanaka, WSC 2001 and 2002
model / shadow / twin taxonomy	Kritzinger et al., 2018
an hour saved at a non-bottleneck is a mirage	Goldratt, Theory of Constraints, 1984

Rule for every future slide and document: a number is either ours with a named source file, or literature with a named reference. If it is neither, it does not appear.

Appendix A — Design rationale on two contested points

Retained for internal reference. Not part of the submission narrative — it argues with a reviewer the reader has not met.

Counterfactual validation is an evaluation instrument, not a pipeline stage. It is tempting to place "validate the candidate by counterfactual" downstream of candidate generation in the live flow. That cannot exist in a plant: you cannot re-run Tuesday morning with identical random draws and one station 30% faster. Counterfactual truth is available only in simulation, offline. Live, the twin commits to a ranked candidate with a confidence and is judged later on realised outcomes. Blurring the two produces a design that cannot be deployed — which is why Part 4.4 emits a counterfactual *comparison* from the model, while Part 6 uses counterfactual *truth* only to score the detector.

Hence the vocabulary rule adopted in Part 4.4: **candidate evaluation** for anything the twin does live, **counterfactual** reserved exclusively for offline scoring. Two words, never interchanged, so no reader can infer we know what would have happened on a shift that cannot be re-run.

"ML readiness" is the wrong axis for this design. Most of this system is deliberately not learned: state reconstruction is arithmetic, constraint ranking is arithmetic, drift detection is 1954 statistics, the buffer countdown is division. Only the horizon-risk model and the failure-mode classifier need training. Scoring the design low on ML readiness penalises exactly the property that makes it defensible — *don't train what you can compute* — and a system that computes what it can compute is easier to explain, cheaper to run, and far harder to attack in a five-minute Q&A.